CONTENTS

SYSTEM REFERENCE · INKPLATE MINIMED MONITOR

How a glucose reading reaches a strip of e-paper

A continuous glucose sensor, an insulin pump, a reverse-engineered cloud client, a self-hosted proxy, and a 2.13″ three-colour display that sleeps for five minutes at a time. This is the whole path, one hop at a time, followed by a full reference to the code that runs on the display itself.

PANEL	WAKE INTERVAL	AWAKE PER CYCLE	DEEP-SLEEP DRAW
212×104px, 3-colour	**~300s**	**~20–45s**	**~8µA**

00 Overview

The device on the caregiver's shelf shows one number, one trend arrow, and a line of insulin — but getting that number there means crossing three separate systems that were never designed to talk to a two-inch e-paper display. The pump computes a glucose reading from its sensor, ships it to Medtronic's cloud, a locally-run proxy pulls it back down onto the home network, and once every five minutes an ESP32 wakes up just long enough to ask for it.

Nothing about this is real-time. The pump's own sensor only produces a new reading every five minutes; the display is built around that fact rather than fighting it, spending most of its life in deep sleep and refreshing only as often as there's genuinely new data to show.

> **Three frontends, one data source.** This document describes the Inkplate 2 e-paper frontend specifically — the code examples and diagrams below are all drawn from its actual `main.py`. A desktop (Tkinter) and an M5Stack Fire frontend exist as siblings, polling the same proxy but with entirely separate, non-shared code.

---

01 System architecture

Five hops separate the sensor under the patient's skin from the display on the wall. Each one exists for a reason — mostly because the systems on either side of it were built by different people, for different purposes, a decade apart.

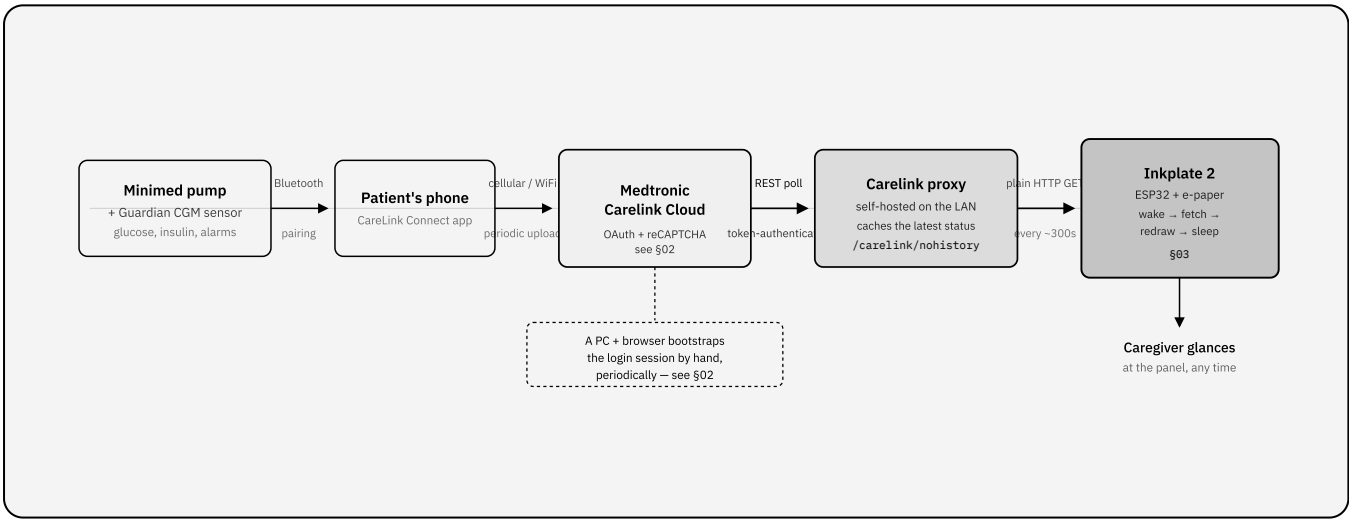

FIGURE 1 · END-TO-END DATA FLOW

Five hops from sensor to screen. The two coloured stages — Carelink Cloud and the Carelink proxy — are the ones this project's own code touches directly; everything to their left is Medtronic's system, unmodified.

The hops, in short

- **Pump + CGM sensor.** The 770G/780G pump reads glucose from a paired Guardian sensor/transmitter roughly every five minutes, runs its own SmartGuard auto-mode logic, delivers insulin, and raises its own alarms (low/high glucose, reservoir, battery, sensor issues) entirely on-device.
- **Patient's phone.** Medtronic's CareLink Connect app pairs with the pump over Bluetooth and periodically uploads its state to Medtronic's own cloud — this hop is

entirely Medtronic's software, outside this project.

- **Carelink Cloud.** Medtronic's server holds the current pump/sensor state behind an authenticated REST API not intended for third-party use — see §02 for what that authentication actually involves.
- **Carelink proxy.** A separately-hosted instance of the Carelink Python Client's proxy, run continuously on the caregiver's own network. It holds the cloud session, refreshes it, and re-exposes the latest status as a trivial local REST endpoint any embedded device can call with a plain HTTP GET.
- **Inkplate device.** Wakes on a timer, polls the proxy, decides what (if anything) needs an alarm, and redraws its panel — the subject of the rest of this document.

---

02 Why a local proxy

The obvious simplification would be to drop the proxy and have the Inkplate talk to Carelink Cloud directly. That was investigated and ruled out — not because of the ongoing polling, but because of what it takes to *start* a session.

STEP	WHAT IT REQUIRES	WHY AN ESP32 CAN'T DO IT
OAuth / social login	A real browser driving Medtronic's login flow	No browser exists on this firmware
reCAPTCHA	A human, solving it	Not automatable, by design
Device registration	A 2048-bit RSA keypair + X.509 CSR	No CSR/X.509 tooling on-device

Every one of those steps happens on a PC, using a real browser (Selenium-driven Firefox), producing a session token. The **proxy's** whole job is to hold onto that token, refresh it, keep polling Carelink Cloud in the background, and expose the result as one unauthenticated, LAN-only endpoint — `GET /carelink/nohistory` — so that every embedded frontend downstream (this device included) never has to know OAuth, CAPTCHAs, or certificates exist.

How the proxy works

`carelink_client2_proxy.py` is a small long-running daemon, typically kept alive via a systemd unit. On startup it constructs a `CareLinkClient` (from the sibling

`carelink_client2.py` library) and calls `.init()`, which reads the token file, resolves the correct regional API config, and fetches the account's user/patient info. If that fails, the proxy's status flips to "Valid token required" and it serves a recovery page instead of data (more on that below) until a human fixes it.

Once initialised, it loops: fetch the latest reading, serve it to whatever polls the HTTP API, then sleep. That sleep isn't a fixed dumb interval — it's calculated from the pump's own last-reported reading time plus the configured wait (300s by default), so the proxy naturally lands its next poll right when a new reading is actually expected, falling back to a 120s retry on error. It's the same "don't poll faster than new data arrives" principle this device's own 5-minute cycle already follows, just applied one hop further up the chain.

Three endpoints, all served from the same process:

ENDPOINT	RETURNS
`GET /`	Status page - or the token-recovery form, if the session has expired
`GET /carelink`	Full data, including the last 24h of history
`GET /carelink/nohistory`	Current snapshot only (what this device polls)

The token file: `logindata.json`

This is the answer to "what does the proxy actually authenticate with?" — `logindata.json` *is* the credential. It's not a password; it's a bundle of OAuth artifacts produced once by the browser-driven login and read by every `CareLinkClient` afterward:

FIELD	ROLE
`access_token`	The short-lived bearer token sent as `Authorization: Bearer …` on every Carelink API call. It's a JWT - the client decodes (not verifies) its own payload locally just to read the `exp` claim and know when it's about to go stale.
`refresh_token`	Exchanged for a fresh `access_token`/`refresh_token` pair when the current one expires - this is what lets the proxy run for days without a human, and itself expires if unused for about a week.
`client_id` / `client_secret`	Identify the registered "device" (the login script, posing as the Carelink Connect Android app) to Medtronic's OAuth server. `client_secret` is only present on the older, non-Auth0 login path - see the diagram below.
`mag-identifier`	A device-registration identifier from the older login path's certificate-based device registration step; sent as a header alongside the bearer token when present.
`scope`	The OAuth scope granted at login - carried through refreshes unchanged.

So yes — this JSON object is exactly "the key the proxy uses": whoever holds a valid `logindata.json` can call Carelink Cloud as that account, indefinitely, without ever seeing the login page again (until the refresh token itself lapses).

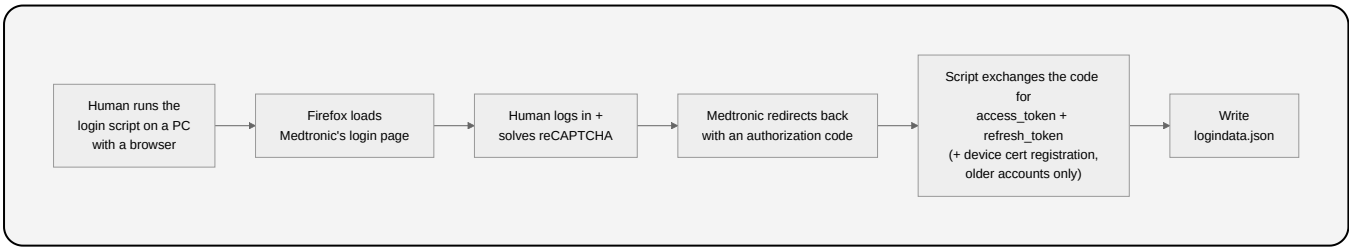

FIGURE 1B · ONE-TIME TOKEN BOOTSTRAP

Everything here needs a human at a real browser, gated by a CAPTCHA - this is the one step no ESP32 (or any headless machine) can do on its own. It only runs once, and again only if the loop below goes fully cold.

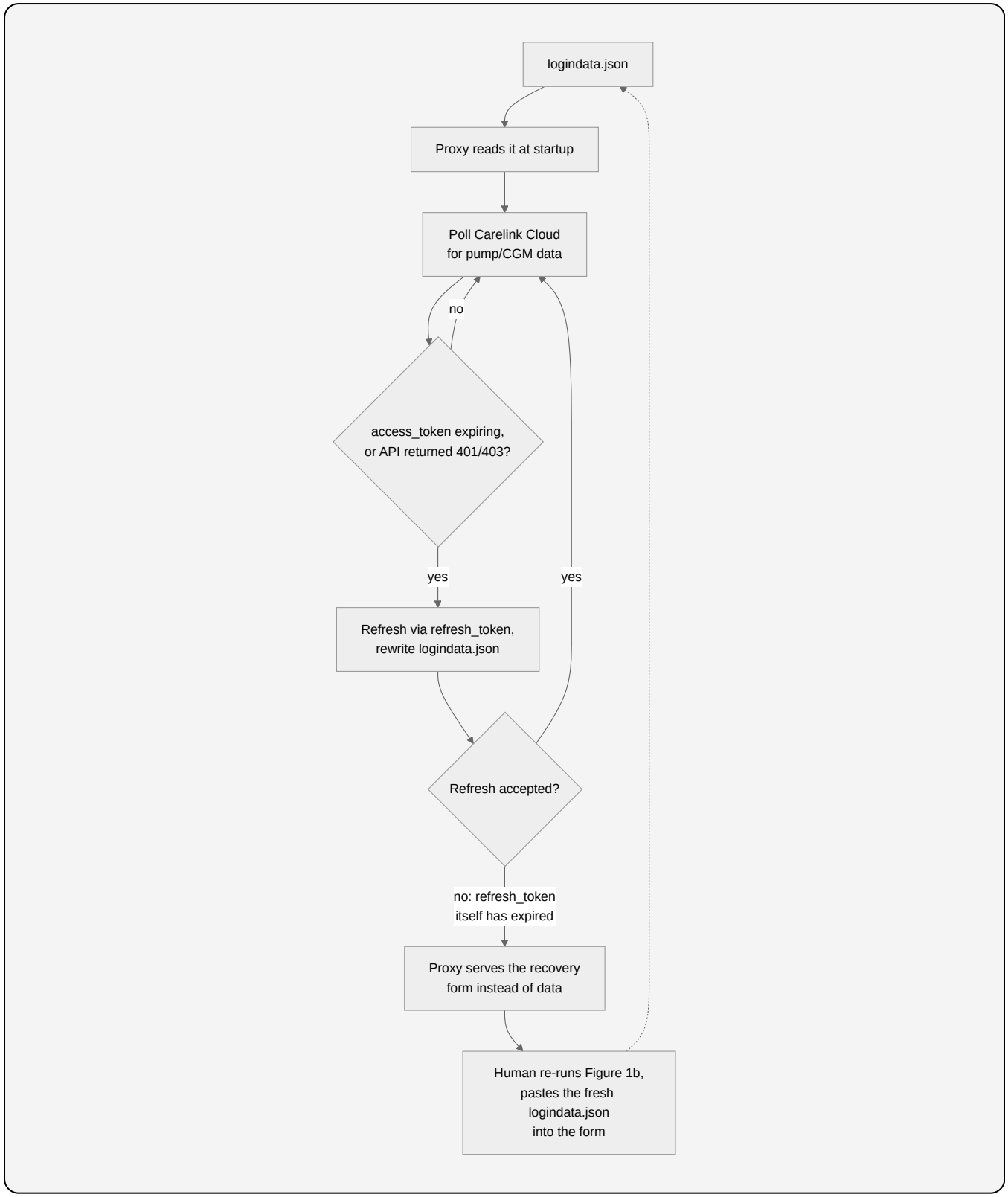

FIGURE 1C · THE PROXY'S DAY-TO-DAY LOOP

No human involved anywhere in this loop except the one recovery branch at the bottom - which only triggers when the refresh token itself has lapsed (roughly: not used for about a week), not on every ordinary token expiry.

What's actually secured, and what isn't

`carelink_client2.py`'s traffic to Medtronic is all HTTPS, and the refresh mechanism above is specifically what avoids ever replaying the CAPTCHA-gated login for routine operation. That's the extent of what it actively secures.

> Neither the data endpoints nor the recovery form require any authentication of their own. The server binds to all network interfaces, not just localhost, and its request handler carries a literal `# Security checks (if any) # TODO` comment. In practice, anyone who can reach the proxy's port can read the patient's live glucose/pump data, or `POST` a replacement token file to it. `logindata.json` itself is also just plain, unencrypted JSON on disk - keeping it out of version control (it's `.gitignore`'d in the source repo) and off any network the patient wouldn't otherwise trust is the operator's responsibility, not something the code enforces. This is a reasonable tradeoff for a single-purpose tool on a home LAN, not a hardened multi-tenant service - worth knowing before running it anywhere less trusted.

> The tradeoff this buys: an always-on machine on the home network to run the proxy, and a human at a PC every so often when the session needs re-bootstrapping. In exchange, the device itself stays as simple as a raw socket and a JSON parser.

---

03 Device operating cycle

`main()` is not a loop — it's a single pass. `machine.deepsleep()` at the end re-runs the entire script from scratch on the next wake, so there is deliberately no state carried in memory between cycles.

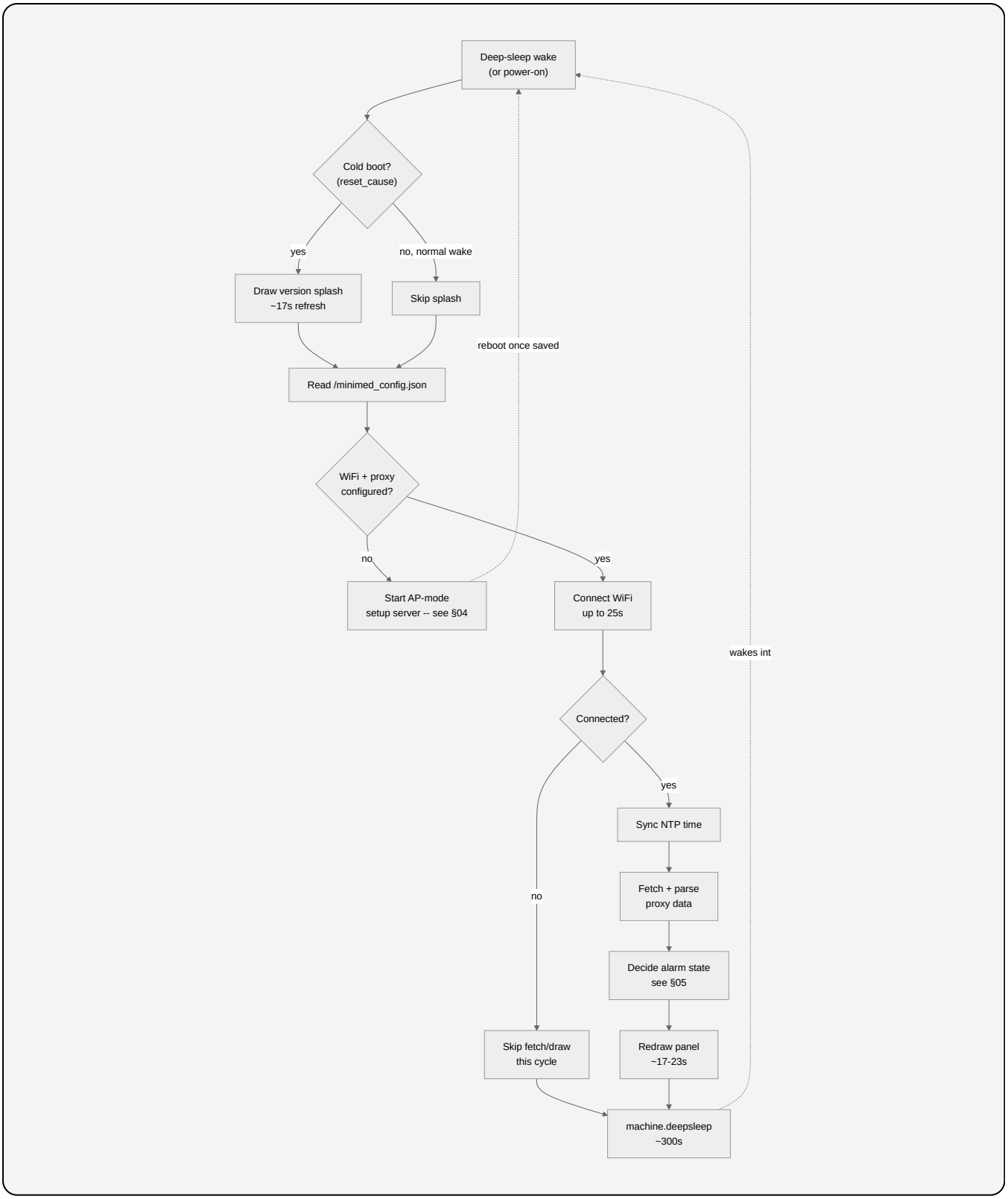

FIGURE 2 · ONE WAKE CYCLE

Every box after "Read config" runs inside a single broad `try/except` (see §08) — any unhandled error anywhere in that block falls straight through to deep sleep rather than crashing to an idle prompt, so a bug costs one missed update, never a stuck device.

Why the radio timeout is generous

Unlike a continuously-powered device that associates with WiFi once and stays connected, this board's radio is fully powered down every cycle by `machine.deepsleep()` and has to cold-reassociate from nothing, every ~5 minutes. A connect failure here does **not** fall through to the setup flow in §04 — that's reserved for a config file that's missing credentials outright. A transient failure on an already-configured device just skips this cycle and tries again next wake.

---

04 First-time setup

The first time the device boots — or any time it can't find stored WiFi credentials — it stops trying to be a display and becomes a tiny access point instead.

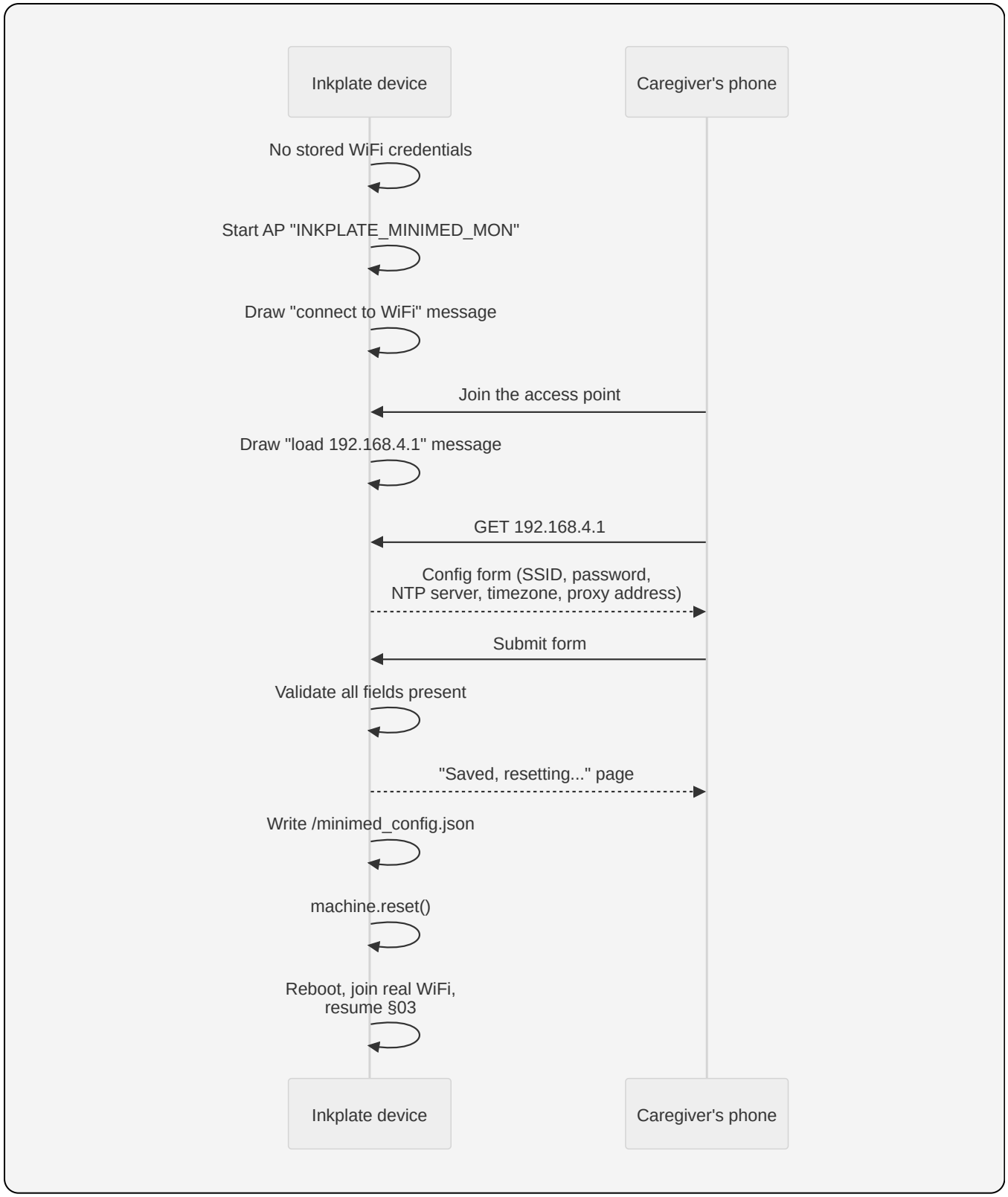

FIGURE 3 · ACCESS-POINT SETUP

The config server (`do_access_point()`) is a hand-rolled HTTP server over a raw socket — the same reason this firmware has no `requests` module also means it has no web framework, so this is ~150 lines of manual request parsing.

05 Alarm decision logic

The proxy's `lastAlarm` field reports the most recent alarm *notification* — it does not update or clear itself just because the underlying glucose reading has since recovered. Deciding whether to actually show a red banner takes two independent checks.

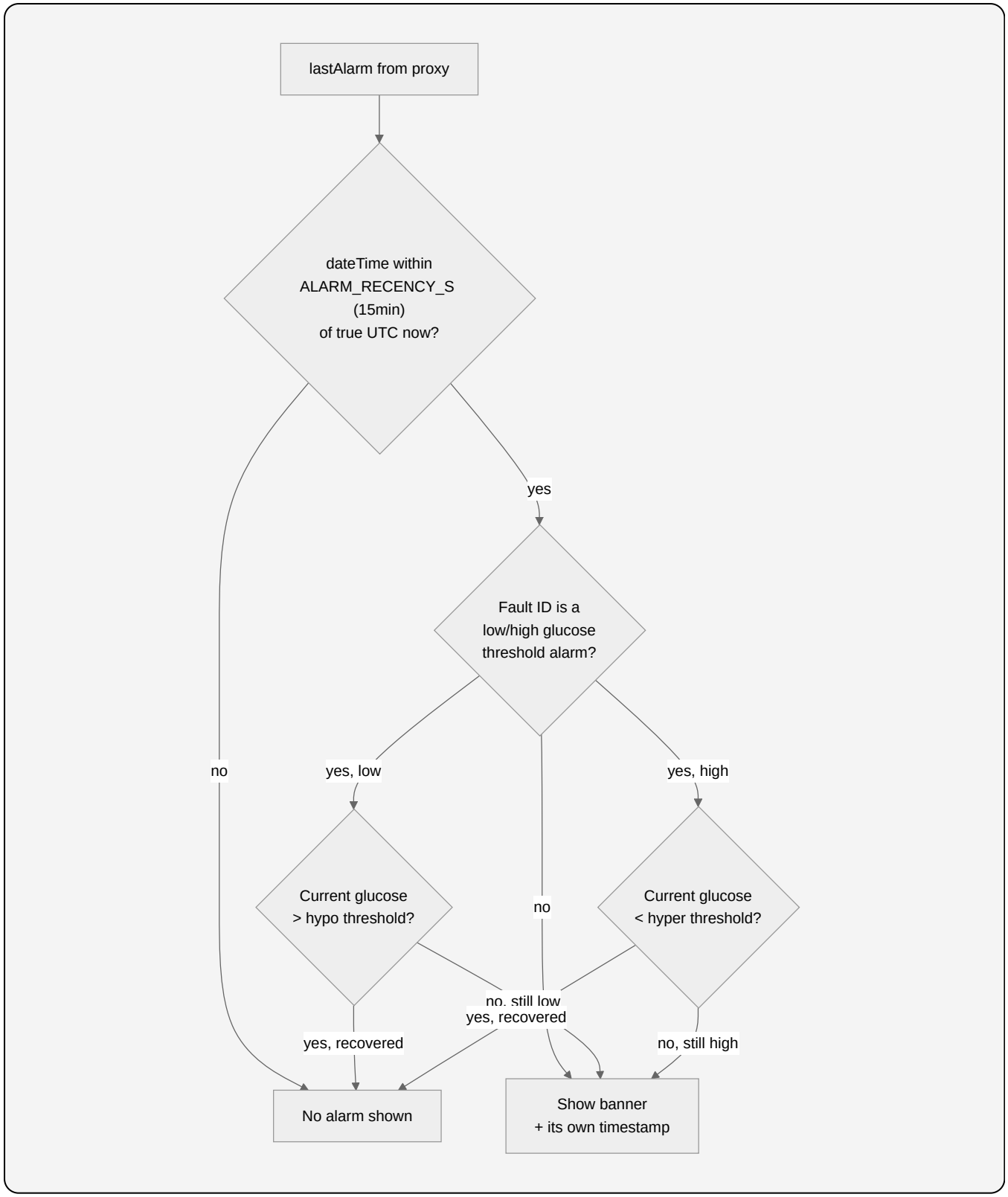

FIGURE 4 · `GET_ALARM_TEXT()`

The recency check alone used to be unreliable: `lastAlarm.dateTime`'s digits turn out to already be local wall-clock time, unlike the proxy's other timestamp field — comparing it directly against true UTC silently stretched the effective window from 15 minutes to roughly two hours. The second check (against the live reading) is what actually catches an alarm clearing early.

Both checks matter independently: without the recency check, a week-old low would still show if the current reading happened to be low again for an unrelated reason. Without the value check, a genuinely-resolved low would keep showing for the rest of its 15-minute window.

Two independent red signals

The glucose figure turning red and the alarm banner appearing are **not the same mechanism**, and can disagree. The number's colour comes from a fixed threshold in this code (`HYPER_THRESHOLD_MGDL`/`HYPO_THRESHOLD_MGDL`, 180/70) applied to whatever the current reading is — nothing more. The banner only appears when Carelink itself reports a `lastAlarm` event, which fires against thresholds configured on the pump by the patient or clinician — values this proxy response doesn't expose, so this code has no way to know or replicate them.

> A reading of 69 mg/dL showing red with *no* banner underneath is expected, not a bug: it crossed this display's own line without necessarily crossing the pump's own configured alarm line. The reverse is just as possible — an in-range reading can still carry an active banner, if the alarm is a non-glucose one (reservoir, battery, sensor) that Figure 4 never corroborates against the current value in the first place.

---

06 Display layout

The panel is 212×104 pixels in three colours, with no partial-refresh capability — every cycle clears and redraws the whole thing in one `display()` call. `draw_screen()` lays out five regions, all computed relative to the glucose number's own height so nothing has to be hand-tuned twice.

white — ground black — text, trend, insulin red — out-of-range value, alarm banner

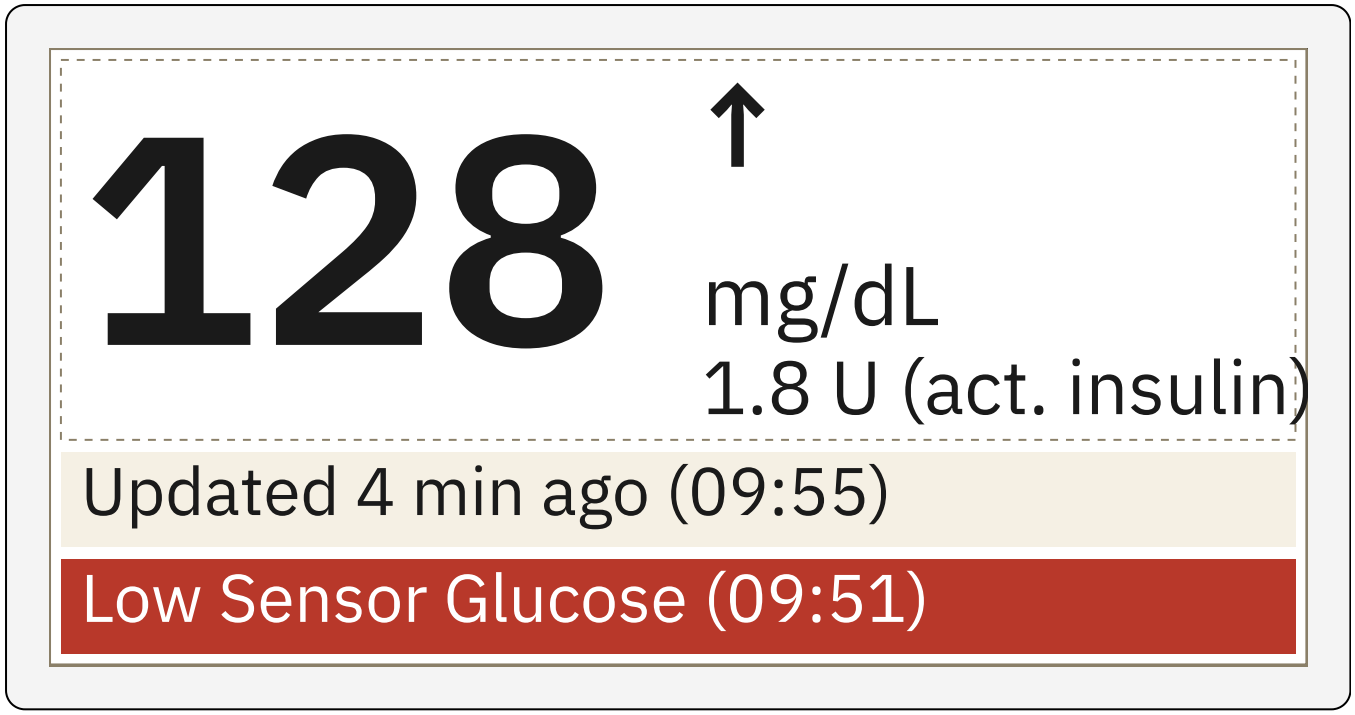

FIGURE 5 · PANEL REGIONS, 3× SCALE

Top-to-bottom: the glucose figure (`gh = FONT_HEIGHT_1X × GLUCOSE_SIZE`, 64px), a right-hand column packed tightly enough that trend + unit + insulin exactly fill that same 64px, then two dedicated bottom rows — the update timestamp always drawn, the alarm banner drawn only when §05 returns one. The two bottom rows used to share a single row; an active alarm used to hide the timestamp entirely.

---

07 Python code reference

Everything above describes behaviour; this section maps it to the ~1,230 lines of `main.py` that actually run on the device — MicroPython, not CPython, with no package manager and no standard library beyond what the firmware ships.

Library calls

Seven imports, all firmware-provided. Two of them replace libraries a normal Python environment would take for granted, because this firmware doesn't ship them.

MODULE	USED FOR	KEY CALLS
`inkplate2`	The e-paper display driver itself	`Inkplate()`, `.begin()`, `.clear_display()`, `.set_cursor()`, `.set_text_size()`/`.set_text_color()`, `.print()`/`.println()`, `.fill_rect()`, `.display()`, `.BLACK`/`.WHITE`/`.RED`, `.font_family.get_ch()`
`ntptime`	Setting the RTC from a network time server	`ntptime.host`, `ntptime.settime()`
`time`	Epochs, wall clocks, elapsed-time timers	`time()`, `localtime()`, `mktime()`, `ticks_ms()`/`ticks_diff()`/`ticks_add()`, `sleep_ms()`
`json`	Parsing the proxy's response; reading/writing config	`loads()`, `dump()`/`load()`
`network`	Both WiFi roles this device plays	`WLAN(STA_IF)` — client, joins the real network; `WLAN(AP_IF)` — the §04 setup access point
`socket`	`no requests module` Raw TCP, in both directions	Outbound: hand-rolled HTTP/1.0 GET to the proxy. Inbound: the §04 config server's own tiny listener
`machine`	Power state and hardware reset info	`deepsleep()`, `reset_cause()`, `DEEPSLEEP_RESET`, `RTC()` (one byte, for the ghosting workaround — §08), `reset()`

Function reference

Grouped by the file's own section order, each mapped back to where it fits in Figure 2's cycle.

FUNCTION	PURPOSE
`config_write() / config_read()`	Persist and load `/minimed_config.json`
`ntp_sync()`	Set the RTC to UTC from the configured NTP server
`local_now()`	Apply timezone + DST offset to the current UTC clock
`http_get()`	Raw-socket HTTP/1.0 GET, returns (status, body)
`web_page_config() / web_page_success()`	The two HTML pages the §04 config server serves
`get_url_param()`	Pull one field out of a raw querystring
`do_ap_status()`	Draw a plain status message during setup (no sound — no speaker)
`do_access_point()`	Run the AP + listen loop end-to-end (Figure 3)
`read_config()`	Load config, falling through to AP mode if credentials are missing
`wlan_connect()`	Join WiFi with a 25s budget; never raises into the caller
`time_delta()`	"X min ago" / "No data" / "Now" phrasing between two timestamps

FUNCTION	PURPOSE
`convert_datetimestr_to_epoch()`	Parse a Carelink ISO-ish datetime string to an epoch value
`getFaultStr()`	Fault ID → canonical ID → human-readable message
`get_alarm_text()`	The full decision in Figure 4
`new_state()`	The empty `state` dict shape every cycle fills in
`handle_pumpdataupdate()`	Fetch, parse, and populate `state` from the proxy's JSON
`_had_banner_last_cycle()` / `_set_banner_flag()`	One-byte RTC-memory flag for the ghosting workaround
`text_width()` / `wrap_text_to_width()` / `truncate_to_width()`	Measure and fit text against this font's real (non-monospace) glyph widths
`draw_screen()`	Lays out Figure 5 and calls `display()`
`main()`	Orchestrates Figure 2, start to finish

08 Design tradeoffs

None of these are oversights — each was a deliberate choice, made once the hardware's actual constraints were understood on real devices rather than in the datasheet.

- **Alarms are visual-only.** This board has no speaker. A caregiver not looking at the panel gets no alert at all — a materially weaker guarantee than an audible alarm, accepted for this device rather than solved.
- **Every redraw is a full flash, not a partial update.** No partial-refresh API exists for this panel, so even a one-digit change repaints all 212×104 pixels, visibly, for 17–23 seconds.
- **Red ink ghosts.** The red plane doesn't fully settle on a single refresh after a large red area (the alarm banner) — clearing it needs one extra blank flash, tracked via a single byte of RTC memory since nothing else survives a deep-sleep restart.
- **No cross-cycle memory, on purpose.** Because `main()` keeps no state, an unhandled exception anywhere in the cycle costs exactly one missed update and self-heals on the next wake — there is no dedup bookkeeping left to corrupt, no flag left stale.
- **No factory-reset gesture yet.** A stuck bad config currently has to be cleared by hand over a serial connection; a power-cycle-counting gesture is designed but not yet wired up.

Inkplate Minimed Monitor · source on GitHub · GPLv3, building on Ondrej Wisniewski's original M5 Minimed Monitor.